

Developer Hub and Platform Engineering

Version 1, 2026-07-10

Introduction

Welcome to this quick course on the ***Developer Hub and Platform Engineering***. This course is the **fifth** in a series of **six** courses about **Application Platform Delivery**:

- *Foundations: Developer Experience and GitOps Workflows*
- *Cloud-Native Development with Dev Spaces*
- *Software Factory and CI/CD*
- *Application Modernization*
- *Developer Hub and Platform Engineering*(This course)
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

This training provides a comprehensive understanding of a modern Software Factory and its role in automating the software delivery lifecycle. Participants will learn how supporting services, Continuous Integration (CI), OpenShift Pipelines (Tekton), Continuous Delivery (CD), and promotion pipelines work together to build, validate, deploy, and promote applications across environments. By the end of the training, learners will understand how to implement standardized, automated, and GitOps-driven software delivery workflows on Red Hat OpenShift, enabling faster, more secure, and reliable application delivery.

Duration: 2.00 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

Developer Hub

Technology companies developing cutting-edge software solutions are increasingly focusing on the **Developer Experience (DevEx)** to improve productivity and reduce time to market. Focusing on improving the developer experience has numerous advantages:

- Reduced cognitive load for developers
- Reduced software development costs
- Increased software delivery speed
- Increased software quality
- Reduced burnout → happier developers → reduced attrition and higher retention rate

Developer Experience is the **user experience** that developers are subject to when developing applications using a platform, along with a host of supporting tools, utilities, and workflows.

The developer experience is a major component of, and usually related to the following terms:

Internal Developer Platform (IDP)

The set of tools and processes that software developers use to create business capabilities, and deploy them in production.

Platform Engineering

A diverse group of technology professionals - developers, system administrators, and site reliability engineers (SRE) that build, run, and maintain an internal developer platform.

Internal Developer Portal (IDP)

A centralized, self-service interface where developers discover services, documentation, standards, ownership, and workflows needed to build and operate software efficiently.

What is Red Hat Developer Hub?

[Red Hat Developer Hub](#) is an enterprise-grade internal developer portal built on the Backstage open source project. It provides a single pane of glass where developers can discover services, create new projects from templates, and access documentation—all without needing to understand the underlying infrastructure.

Why an Internal Developer Portal?

As organizations adopt microservices, GitOps, and platform engineering, developers face increasing complexity: multiple Git repositories, CI/CD tools, deployment targets, and observability platforms. An internal developer portal addresses this by providing:

- **Discoverability**—A searchable software catalog of all services, APIs, and infrastructure components

- **Self-service** — Golden Path templates that let developers create new projects with all the scaffolding pre-configured
- **Consistency** — Every new project follows organizational standards for CI/CD, security, and observability
- **Reduced cognitive load** — Developers interact with one portal instead of navigating dozens of tools

Architecture

Developer Hub has several core components:

Software Catalog

A centralized registry of all software components (services, APIs, libraries, infrastructure) in the organization. Each component has an owner, lifecycle stage, and links to its repository, CI pipelines, and documentation.

Golden Path Templates

Parameterized project scaffolds that generate complete, ready-to-deploy projects. A template typically creates a Git repository, CI pipeline, GitOps manifests, `devfile`, and catalog registration — all in one click. They are also called **Software Templates**.

Plugins

Extend Developer Hub with integrations for external tools. Common plugins include Tekton (pipeline status), Argo CD (deployment status), Topology (OpenShift view), and Dev Spaces (launch workspace).

TechDocs

Documentation-as-code that renders Markdown docs alongside catalog entries. Developers find API documentation, runbooks, and architecture decisions in the same portal where they manage their services.

Key Concepts

Software Catalog Entity

A YAML file (`catalog-info.yaml`) at the root of a repository that registers the component in the catalog. Defines metadata like name, owner, type, lifecycle, and relationships to other components.

Golden Path Template

A `template.yaml` that defines parameters (user inputs), steps (actions to execute), and outputs (what gets created). Steps can create repositories, render files from skeletons, register catalog entities, and trigger pipelines.

Plugin

A frontend or backend module that adds functionality to the portal. Red Hat Developer Hub ships with curated plugins for OpenShift ecosystem tools.

How Developer Hub Fits the Software Factory

Developer Hub ties together all the tools from Day 2 into a unified developer experience:

Tool	Developer Hub Integration
Dev Spaces	Launch a workspace directly from a catalog component
Tekton	View pipeline runs and status for each component
Argo CD	See deployment status across environments
GitLab	Link to source repositories and merge requests
OpenTelemetry	Access observability dashboards from the component page

A developer's workflow becomes:

1. Open Developer Hub
2. Create a new project from a Golden Path template
3. Launch a Dev Spaces workspace from the catalog
4. Write code, push, and watch the pipeline run — all visible in the portal

Next Steps

When you are ready for the hands-on lab, proceed to [Environment Overview](#) to get started.

Install and configure Red Hat Developer Hub

After the Software Factory work in Lab 1, you need a single front door where developers discover services, templates, and links to the tools they use every day. Red Hat Developer Hub provides that experience on OpenShift when you pair the Red Hat Developer Hub Operator with a small `app-config` and a `Backstage` custom resource.

Red Hat Developer Hub is a component of Red Hat Advanced Developer Suite that enables platform engineers to build internal developer portals for their organization. It is based on the `Backstage` project for building internal developer portals. This project was donated to the `CNCF` by Spotify in 2020. Platform engineers can use Red Hat Developer Hub to build internal developer portals. Doing so involves integrating Red Hat Developer Hub with various data sources, cataloging existing software components, infrastructure, and resources, configuring authentication, and more.

Learning objectives

By the end of this lab, you will be able to:

- Install the Red Hat Developer Hub Operator on your cluster.
- Create a project, supply a minimal `app-config` through a ConfigMap, and deploy a `Backstage` custom resource.
- Verify pods, routes, and sign-in to Red Hat Developer Hub using the guest provider configured

for lab use.

- Explain why guest authentication is restricted to non-production scenarios.

Exercise 1: Install the Operator and create the **setup-rhdh** project

1. Open the OpenShift web console at {openshift_console_url} and sign in with {user} and {password}.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Red Hat Developer Hub**.



Make sure that you choose the **Operator Backed** entry from the results.

4. Click btn:[Install]
5. Keep the operator installation namespace as **rhdh-operator**
6. Accept the defaults for the rest (All namespaces, Automatic approval) and click btn:[Install]
7. Wait for the operator to reach the **Succeeded** phase

From the terminal tab, create the **setup-rhdh** the project:

```
oc create namespace setup-rhdh
```

Expected output (namespace line may vary slightly by cluster):

```
Now using project "setup-rhdh" on server "https://api.openshift.example.com:6443".
```

Exercise 2: Create the **rhdh-config** ConfigMap

Red Hat Developer Hub reads **app-config** settings from files mounted into the application container. You store that file in a ConfigMap in the **setup-rhdh** namespace.

1. In the console, open **Workloads** and **ConfigMaps** while **setup-rhdh** is selected.
2. Click **Create ConfigMap**, switch to **YAML view**, replace the placeholder content, and apply the following manifest.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: rhdh-config
  namespace: setup-rhdh
data:
  app-config.yaml: |
    app:
      title: Red Hat Developer Hub
    backend:
```



```

    reading:
      allow:
        - host: 'gitlab.{openshift_cluster_ingress_domain}'
  auth:
    providers:
      guest:
        dangerouslyAllowOutsideDevelopment: true

```

The `backend.reading.allow` entry permits the Developer Hub backend to fetch catalog data and templates from `{gitlab_url}`.



Guest access with `dangerouslyAllowOutsideDevelopment` is only for isolated lab environments. Do not use this pattern when the cluster reaches real data sources or production systems.

3. Click **Create** to save the ConfigMap.

Verify

```
oc get configmap rhdh-config -n setup-rhdh -o yaml
```

Expected output (truncated):

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: rhdh-config
  namespace: setup-rhdh
...

```

Exercise 3: Deploy the **Backstage** custom resource

The Red Hat Developer Hub Operator watches for **Backstage** resources and reconciles PostgreSQL and application pods, routes, and mounts for your ConfigMap. The `spec.application.appConfig` section mounts `rhdh-config` into the Backstage container so the running instance loads `app-config.yaml`, including `backend.reading.allow` for the GitLab host at `{gitlab_url}`.

1. Click the plus (+) icon in the top bar and choose **Import YAML**.
2. Paste the following manifest and select your **setup-rhdh** project if prompted.

```

apiVersion: rhdh.redhat.com/v1alpha3
kind: Backstage
metadata:
  name: rhdh
  namespace: setup-rhdh
spec:

```

```
application:
  appConfig:
    mountPath: /opt/app-root/src
    configMaps:
      - name: rhdh-config
  route:
    enabled: true
  database:
    enableLocalDb: true
```

3. Click **Create**.

Verify

Watch the pods until PostgreSQL and the main Backstage pod report **Running** and ready:

```
oc get pods -n setup-rhdh
```

Expected output (names and ages will differ):

NAME	READY	STATUS	RESTARTS	AGE
backstage-rhdh-xxxxxxxx-xx	1/1	Running	0	2m
postgresql-xxxxxxxx-xx	1/1	Running	0	3m

Check the custom resource and route:

```
oc get backstage -n setup-rhdh
```

```
oc get route backstage-rhdh -n setup-rhdh -o jsonpath='https://{.spec.host}{"\n"}'
```

Expected route list includes an entry for the Developer Hub host with port 443.

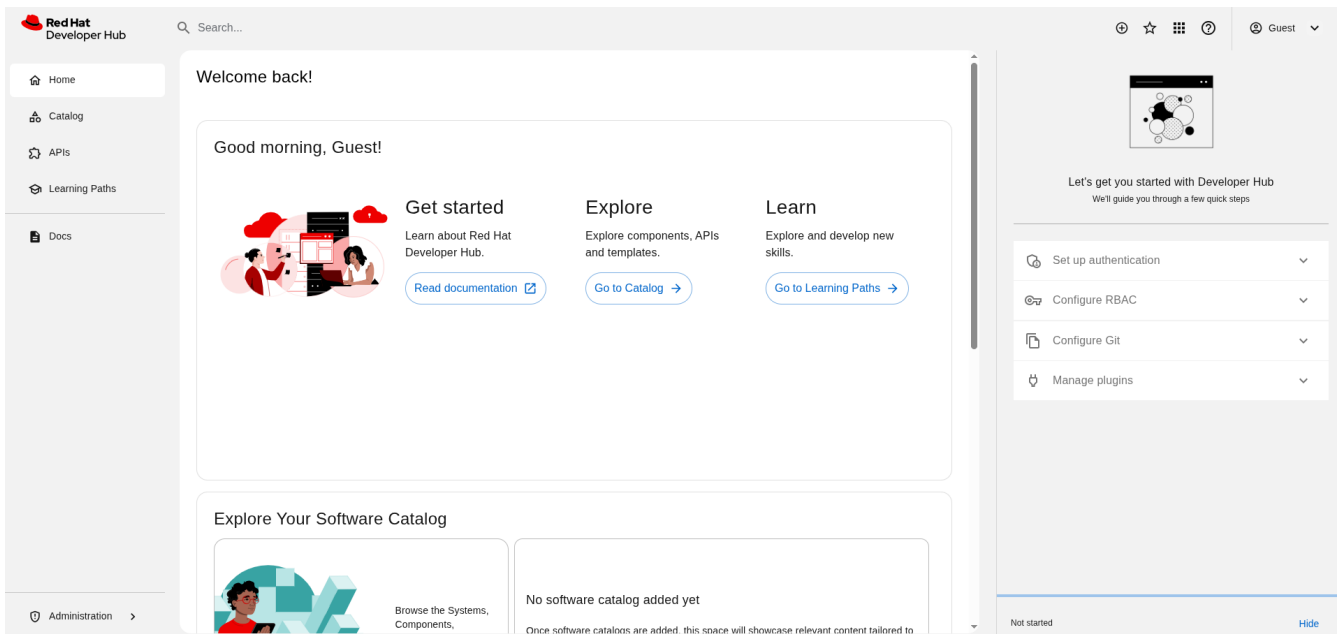
Exercise 4: Open Red Hat Developer Hub and sign in

1. Open **Networking** and **Routes** for project **setup-rhdh**.
2. Click the **Location** URL for the Developer Hub route, or use the **Developer Hub** tab in the Showroom UI if it is configured for your environment.

If you see **Application is not available**, wait one minute and refresh. Pods may still be completing readiness checks.

3. On the sign-in screen, choose **Guest**.

You should land on the home page signed in as a guest user.



Verify

Confirm you can load the UI and that the browser shows Red Hat Developer Hub with **Guest** context.

Troubleshooting

Issue: Pods stay in `CrashLoopBackOff` for the Backstage workload. **Solution:** List pods with `oc get pods -n setup-rhdh`, run `oc describe pod <pod-name> -n setup-rhdh` on the Backstage pod, and confirm the ConfigMap mounted without typos. Ensure `app-config` URLs match your route hostname.

Issue: Route returns 503 or application not available. **Solution:** Wait for readiness probes to pass. Check `oc get pods -n setup-rhdh` until both pods are ready.

Learning outcomes checkpoint

Before you leave this lab, confirm you can:

- ☐ Explain how a portal complements the Software Factory capabilities from Lab 1.
- ☐ List the main resources you created (`Project`, `ConfigMap`, `Backstage` CR) and what each one does.
- ☐ Contrast guest access with enterprise authentication for real environments.

Summary

You deployed Red Hat Developer Hub using the Operator, a minimal `app-config` in a ConfigMap, and a `Backstage` custom resource in the `setup-rhdh` namespace.

Next steps

Connect Red Hat Developer Hub to Git providers, identity providers, and GitOps so catalogs and templates reflect your toolchain. See [Red Hat Developer Hub documentation](#) for supported

integrations and security guidance.

For Golden Path templates, Dev Spaces integration, and authentication configuration, continue with [Developer Hub: templates and integrations](#).

Developer Hub: templates and integrations

Complete [Install and configure Red Hat Developer Hub](#) before this section so you have a running instance.

Overview

In this section you will register a Golden Path or Software template that scaffolds new Quarkus projects with CI/CD pre-configured, integrate Dev Spaces for one-click workspace creation, and configure OpenShift OAuth for enterprise authentication.

Software templates in Red Hat Developer Hub enable your team(s) to create Entities, such as new Components, and - through the use of "actions" provided by plugins - create resources in other systems such as your GitLab, GitHub, and OpenShift GitOps instances. Templates themselves are represented as Entities in the Software Catalog, meaning you can import them similar to any other Entity!

Platform Engineers will often be the authors of Templates, and use them to create "golden paths" that follow best-practices and use approved processes and tooling. Development teams will be the consumers of Templates to create new software and automate their tasks. Using Templates reduces cognitive load on the development teams by allowing them to focus on development tasks, while platform concerns are addressed by the template.

Templates are defined using YAML, but are rendered as a rich form in the Red Hat Developer Hub UI when used by development teams.

Register a Golden Path Template

Golden Path templates provide a self-service way for developers to create new projects that follow organizational standards.

Template Structure

A Golden Path template consists of:

```
golden-path-quarkus/
├── template.yaml      # Template definition
├── skeleton/
│   ├── devfile.yaml   # Dev Spaces workspace config
│   ├── pom.xml        # Quarkus project with dependencies
│   ├── src/           # Application source code
│   ├── catalog-info.yaml # Backstage catalog entity
│   ├── tekton/        # Pipeline definitions
│   └── pipeline.yaml
```

```

|   └── trigger.yaml
└── gitops/           # Argo CD manifests
    ├── dev/
    │   ├── deployment.yaml
    │   ├── service.yaml
    │   └── route.yaml
    └── stage/
        ├── deployment.yaml
        ├── service.yaml
        └── route.yaml

```

Create the Template Definition

Create a Git repository in GitLab to hold the template definition.:

1. Log into GitLab:

```
{gitlab_url}
```

2. Navigate to the Software Factory group
3. Create a new empty and public project: **golden-path-quarkus**
4. Uncheck the "Initialize repository with a README" checkbox.
5. Clone it locally:

```

cd ~/workshop
git clone {gitlab_url}/software-factory/golden-path-quarkus.git
cd golden-path-quarkus

```

Create **template.yaml** in your Git repository:

```

apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: quarkus-golden-path
  title: Quarkus Application (Golden Path)
  description: |
    Create a Quarkus application with CI/CD pipeline, GitOps deployment,
    Dev Spaces workspace, and OpenTelemetry instrumentation pre-configured.
  tags:
    - quarkus
    - java
    - recommended
spec:
  owner: platform-team
  type: service
  parameters:

```

```

- title: Application Details
  required:
    - name
    - owner
  properties:
    name:
      title: Application Name
      type: string
      pattern: "^[a-z0-9-]+$"
    description:
      title: Description
      type: string
    owner:
      title: Owner
      type: string
      ui:field: OwnerPicker

- title: Repository
  required:
    - repoUrl
  properties:
    repoUrl:
      title: Repository Location
      type: string
      ui:field: RepoUrlPicker
      ui:options:
        allowedHosts:
          - {gitlab_url}

steps:
- id: fetch-skeleton
  name: Fetch Skeleton
  action: fetch:template
  input:
    url: ./skeleton
    values:
      name: ${{ parameters.name }}
      description: ${{ parameters.description }}
      owner: ${{ parameters.owner }}

- id: publish
  name: Publish to GitLab
  action: publish:gitlab
  input:
    repoUrl: ${{ parameters.repoUrl }}
    description: ${{ parameters.description }}
    defaultBranch: main

- id: register
  name: Register in Catalog
  action: catalog:register
  input:

```

```
repoContentsUrl: ${ steps['publish'].output.repoContentsUrl }}
catalogInfoPath: /catalog-info.yaml
```

```
output:
  links:
    - title: Repository
      url: ${ steps['publish'].output.remoteUrl }}
    - title: Open in Catalog
      icon: catalog
      entityRef: ${ steps['register'].output.entityRef }}
```

Create the Catalog Entity

Add `catalog-info.yaml` to the skeleton directory:

```
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: ${ values.name }}
  description: ${ values.description }}
  annotations:
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: ${ values.name }}-dev
    janus-idp.io/tekton: ${ values.name }}
spec:
  type: service
  lifecycle: production
  owner: ${ values.owner }}
```

Commit and push the template repository:

```
git add .
git commit -m "feat: initial commit"
git push origin main
```

Register the Template in Developer Hub

Red Hat Developer Hub discovers Software Templates through **catalog locations** in `app-config.yaml`. After you push `template.yaml` to GitLab, add a `catalog` section that points at that file so the catalog importer loads your Golden Path template. You update the existing `rhdh-config` ConfigMap in the `setup-rhdh` namespace—the same ConfigMap created in [Install and configure Red Hat Developer Hub](#).

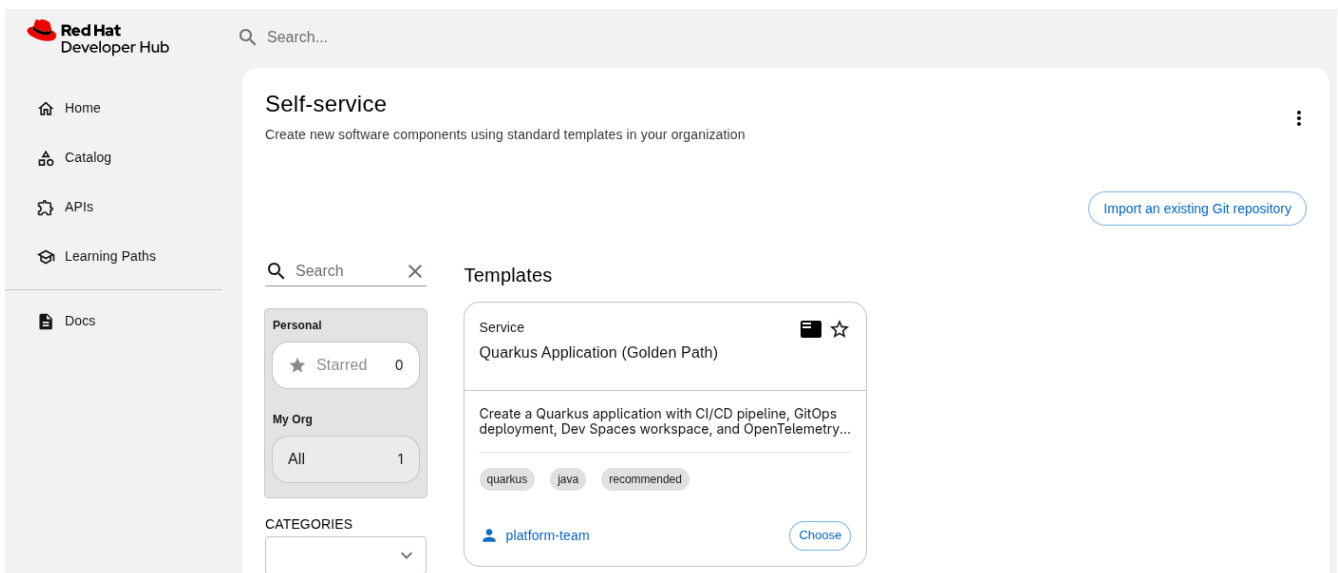
1. Open `{openshift_console_url}[^]` and switch to the **setup-rhdh** project.
2. In the console, open **Workloads** → **ConfigMaps** and select **rhdh-config**.
3. Click **Actions** → **Edit ConfigMap** and stay in **YAML view** (or open the **Data** tab and edit the `app-config.yaml` key).

4. Under `data.app-config.yaml`, keep your existing `app` and `auth` settings and add the `catalog` block below. Do not replace the whole file—merge the new section with what is already there.

```
catalog:
  locations:
    - type: url
      target: '{gitlab_url}/software-factory/golden-path-quarkus/-/raw/main/template.yaml'
  rules:
    - allow: [Template]
```

5. Click **Save** to apply the ConfigMap.

The Red Hat Developer Hub Operator rolls out the change and restarts the application pod. When the pod is **Running** again, open Developer Hub and confirm the template appears under menu:Create[Templates].



Integrate with Dev Spaces

Add a Dev Spaces link to catalog components so developers can open a workspace directly from the component page.

Add the Dev Spaces URL Annotation

In the `catalog-info.yaml` skeleton, add the Dev Spaces annotation:

```
metadata:
  annotations:
    che.eclipse.org/che-server: {devspaces_url}
```



Do not forget to commit and push the changes to the Git repository.


```
git add catalog-info.yaml
git commit -m "feat: add dev spaces annotation"
git push origin main
```

When a component has this annotation, the Developer Hub UI shows a "Dev Spaces" link that opens a workspace for the component's repository.

Configure the Dev Spaces Plugin

Add the Dev Spaces plugin configuration to **rhdh-config** ConfigMap. Under **data.app-config.yaml**, keep your existing **app**, **auth** and other settings and add the **devSpaces** block below. Do not replace the whole file—merge the new section with what is already there.

```
devSpaces:
  defaultUrl: {devspaces_url}
```

Update the ConfigMap and wait for the pod to restart.

Configure OpenShift OAuth Authentication

Replace guest authentication with OpenShift OAuth for production-like access.

Create an OAuth Client

```
oc create -f - <<EOF
apiVersion: oauth.openshift.io/v1
kind: OAuthClient
metadata:
  name: rhdh-oauth-client
grantMethod: auto
redirectURIs:
  - https://backstage-rhdh-setup-
    rhdh.{openshift_cluster_ingress_domain}/api/auth/oidc/handler/frame
secret: GENERATE_A_RANDOM_SECRET
EOF
```



You can generate a random secret value using the following command:

```
node -p 'require("crypto").randomBytes(24).toString("base64")'
```

Create the Auth Secret

```
oc create secret generic rhdh-auth-secret -n setup-rhdh \
  --from-literal=AUTH_OIDC_CLIENT_ID=rhdh-oauth-client \
  --from-literal=AUTH_OIDC_CLIENT_SECRET=GENERATE_A_RANDOM_SECRET \
```

```
--from-literal=AUTH_OIDC_METADATA_URL=https://oauth
-openshift.{openshift_cluster_ingress_domain}/.well-known/openid-configuration
```

Update app-config

Replace the guest auth block with OIDC:

```
signInPage: oidc
auth:
  session:
    secret: MY_SESSION_SECRET
  environment: production
  providers:
    oidc:
      production:
        clientId: ${AUTH_OIDC_CLIENT_ID}
        clientSecret: ${AUTH_OIDC_CLIENT_SECRET}
        metadataUrl: ${AUTH_OIDC_METADATA_URL}
        prompt: auto
```



You can replace `MY_SESSION_SECRET` with a random value using the following command:

```
node -p 'require("crypto").randomBytes(24).toString("base64")'
```

Click **Save** and wait for the Backstage pod to restart.

`signInPage: oidc` shows the OpenShift sign-in flow on the login page. The `${AUTH_OIDC_*}` placeholders are resolved from the Secret mounted on the **Backstage** CR in the next step.

Mount the Secret in the Backstage CR

Mount `rhdh-auth-secret` on the **Backstage** custom resource so the OIDC client ID, secret, and metadata URL are available to `app-config` as environment variables.

1. Open `{openshift_console_url}[^]` and switch to the **setup-rhdh** project.
2. Go to **Ecosystem** → **Installed Operators**.
3. Select **Red Hat Developer Hub**.
4. Open the **Backstage** tab.
5. Click the **rhdh** instance created.
6. Click **Actions** → **Edit Backstage** and switch to **YAML view**.
7. Under `spec.application`, add the `extraEnvs` block below. Keep the existing `appConfig` and `route` sections from the initial deployment.

```
extraEnvs:
  secrets:
    - name: rhdh-auth-secret
```

8. Click **Save**.

The Red Hat Developer Hub Operator reconciles the CR and restarts the Backstage pod.

Verify OpenShift sign-in

1. Wait until the Backstage pod in **setup-rhdh** is **Running**.
2. Open the Developer Hub route URL from the first step in this section.
3. On the login page, choose **OIDC**.
4. Sign in with your OpenShift credentials (for example **{user}** / **{password}**).

You should land on the Developer Hub home page authenticated as your OpenShift user, not as Guest.



The signed-in user must exist in the Software Catalog (or be mapped via your org setup) to use some catalog features. For template creation in this lab, guest access is no longer available after this change.

Test the Golden Path

1. Open Developer Hub and go to menu:Create[Templates]
2. Select the **Quarkus Application (Golden Path)** template
3. Fill in the parameters (name, owner, repository location)
4. Click btn:[Create]
5. Verify the template:
 - A new GitLab repository is created with the skeleton code
 - The component is registered in the Developer Hub catalog
 - The repository includes a devfile, Tekton pipeline, and GitOps manifests
6. Open the component in the catalog and click the Dev Spaces link to launch a workspace

Summary

Developer Hub now provides:

- **Golden Path templates** that scaffold complete projects with CI/CD, GitOps, and Dev Spaces pre-configured
- **Software Catalog** with component metadata, ownership, and links to pipelines and deployments
- **Dev Spaces integration** for one-click workspace creation from catalog components

- **OpenShift OAuth** for enterprise authentication

Next Steps

Proceed to [Day 2 Recap](#).